

Privacy Defenses in LLMs

ELL8299 · COL8383

Tanmoy Chakraborty
Professor, IIT Delhi
<https://tanmoychak.com/>



From attacks to defenses

Attack	What We Learned
Memorization & Extraction	LLMs verbatim regurgitate training sequences when prompted
Membership Inference	An adversary can determine with high confidence if you were in the training set
Prompt & ICL Leakage	System prompts and demonstrations are recoverable from model outputs
Prompt Injection	Malicious inputs can override intended model behavior

Heuristic defenses are unprovable.

Today we study defenses that offer **formal, quantifiable guarantees**.



Why heuristics are not enough?

What heuristic defenses give us:

- **Redaction removes named entities**, but context still leaks identity
- **Output filtering blocks some extractions**, but not all, and it can be bypassed.
- **Deduplicating the training data reduces memorization**, but we still can't say how exposed each record is.

None of these gives us number we can report, audit, or prove.



Why heuristics are not enough?

What a formal guarantee gives us:

- **A quantified bound on information leakage:** the privacy budget ϵ .
- **Adversary-agnostic guarantee:** the proof holds against all adversaries.
- **Composability:** we can reason about what happens when defences are combined.
- **Accountability:** regulators, auditors, and users can verify the claim.

Differential Privacy is the framework that makes this possible.

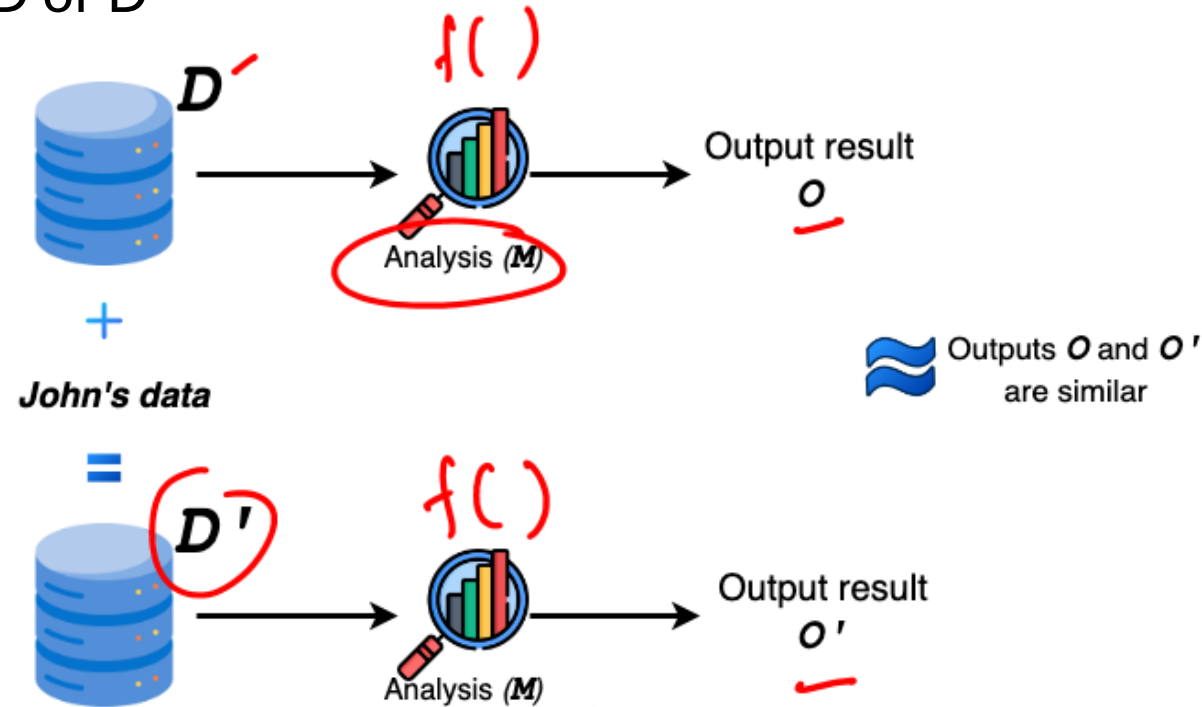


Differential Privacy as a Defence

Formal guarantees against memorization, extraction and inference attacks

Differential privacy: Intuition

- Two datasets D and D' are neighbors if they differ in exactly one person's record
- Differential Privacy (DP) says that a randomized algorithm's output looks almost the same whether it ran on D or D'



<https://ealizabeth.com/blog/abc-of-differential-privacy/>



Differential privacy: Example

ID	Name	Age	Disease
1	Alice	25	Diabetes
2	Bob	31	Flu
3	Carol	42	Diabetes
4	David	37	Cancer
5	Eve	29	Diabetes

$$f(D) = 3$$

$F(D)$ = How many patients have diabetes?

Suppose an attacker already knows Bob, Carol, David, and Eve's records. They also know Alice may or may not be in the database.

If the system says **3**, the attacker immediately learns something about Alice: Alice has diabetes

ID	Name	Age	Disease
2	Bob	31	Flu
3	Carol	42	Diabetes
4	David	37	Cancer
5	Eve	29	Diabetes

$$f(D') = 2$$



Differential privacy: Definition

A randomized mechanism M is (ϵ, δ) -DP, for all neighbours D and D' and every set of outcomes O , if

$$\Pr[M(\check{D}) \in \underline{O}] \leq \exp(\epsilon) \times \Pr[M(\check{D}') \in \underline{O}] + \delta$$

Handwritten annotations: A red '0' is written above the epsilon symbol. A red 'epsilon' is written above the exp function. A red 'epsilon = 0' is written above the epsilon symbol. The entire equation is underlined in red. A red circle is drawn around the exp function. A red checkmark is above the D in M(D). A red slash is at the end of the equation.

- ϵ is the privacy budget. Lower value $\Rightarrow$ more privacy, lesser utility. [Privacy utility trade off](#)
- δ is the failure probability. Typically set to $< 1/N$
- At $\delta = 0$, the mechanism is ϵ -DP
- Think of ϵ and δ as two ways of providing slack in this inequality. If you insist that ϵ be extremely small, the required δ can be larger.



Differential privacy for LLMs

DP can *theoretically* be applied to any of the surfaces where private data enters

- Training data: Train with DP-SGD
- User's shared text: Apply DP-based text sanitization



DP-SGD

Training with privacy guarantees

Privacy problems with SGD

- **Unbounded influence:** Since the gradient of a **single outlier** can be arbitrarily large, an individual datapoint can exert unlimited influence on the model during training.
- **Deterministic leakage:** Without noise, gradients are a deterministic function of the batch, allowing for potential **reconstruction of private training data** via inversion attacks (search for an input whose gradient matches the observed gradient).

DP-SGD attempts to fix these problems by:

- Clipping gradients to bound influence ✓
- Adding Gaussian noise to gradients ✓



Incorporating differential privacy in deep learning

Deep Learning with Differential Privacy

October 25, 2016

Martín Abadi*

H. Brendan McMahan*

Andy Chu*

Ilya Mironov*

Li Zhang*

Ian Goodfellow†

Kunal Talwar*

*Google.

†OpenAI. Work done while at Google.

<https://arxiv.org/pdf/1607.00133>



Training with DP-SGD

1. Compute per-example gradients for every sample x_i

$$\mathbf{g}_t(x_i) \leftarrow \nabla_{\theta_t} \mathcal{L}(\theta_t, x_i)$$

2. Clip gradients

$$\bar{\mathbf{g}}_t(x_i) \leftarrow \mathbf{g}_t(x_i) / \max\left(1, \frac{\|\mathbf{g}_t(x_i)\|_2}{C}\right)$$

3. Add noise and average

$$\tilde{\mathbf{g}}_t \leftarrow \frac{1}{L} \left(\sum_i \bar{\mathbf{g}}_t(x_i) + \mathcal{N}(0, \sigma^2 C^2 \mathbf{I}) \right)$$

4. Update parameters

$$\theta_{t+1} \leftarrow \theta_t - \eta_t \tilde{\mathbf{g}}_t$$

- Clipping limits how much information one individual can contribute.
- Noise makes it difficult to determine whether that individual's contribution was present at all.



How clipping and noise give privacy

Clipping

- Gaussian noise gives (ϵ, δ) -DP only if sensitivity (maximum change in gradient w.r.t one training data) is bounded
- A raw gradient has no bound, clipping forces $\|\bar{g}(x_i)\|_2 \leq C \Rightarrow \text{sensitivity} = C$

Noise Addition

- Noise is scaled to the sensitivity: $N(0, \sigma^2 C^2 I)$, bigger C demands proportionally bigger noise
- σ (noise multiplier) controls the privacy level: larger $\sigma \rightarrow$ smaller ϵ (one step: $\sigma \geq \sqrt{2 \ln(1.25/\delta) / \epsilon}$)

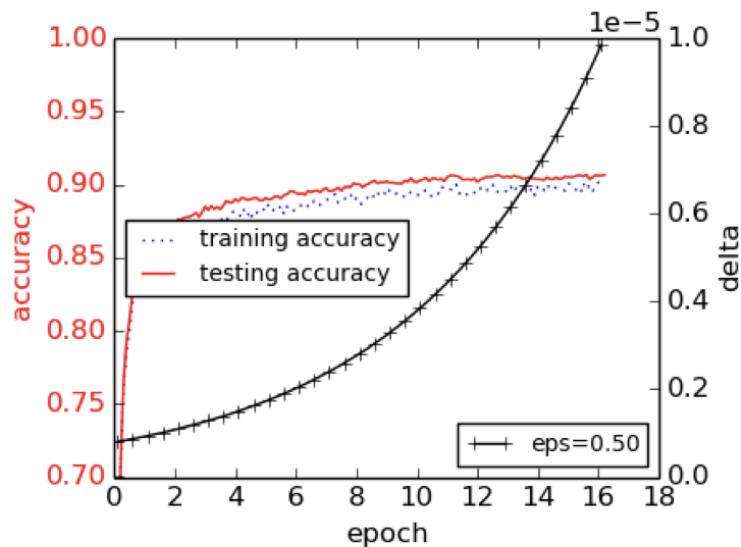






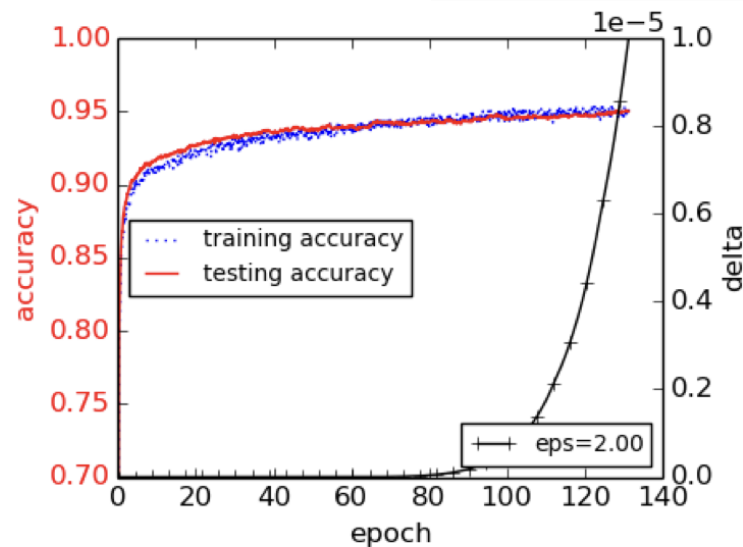
DP-SGD: Results on MNIST

Privacy $\uparrow \iff$ Noise $\uparrow \iff$ Utility/Accuracy $\downarrow$



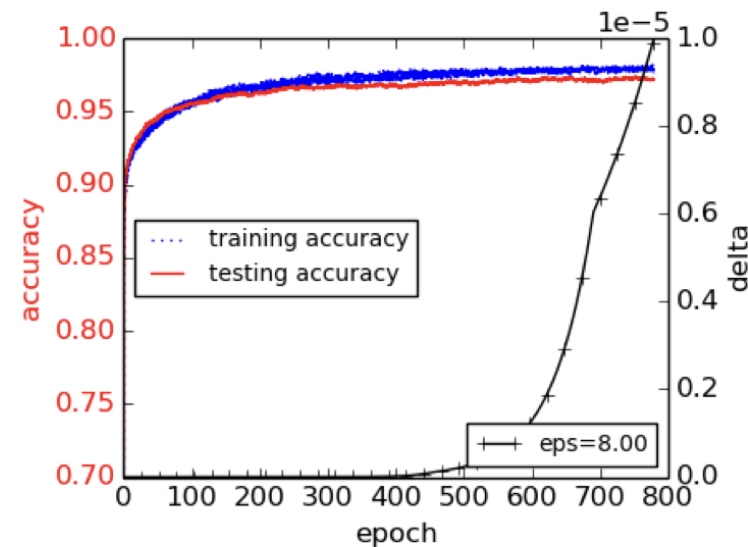
(1) Large noise

Training stops after only around **16 epochs** before the specified privacy budget is exhausted.



(2) Medium noise

Roughly **130 epochs** under the plotted privacy accounting.



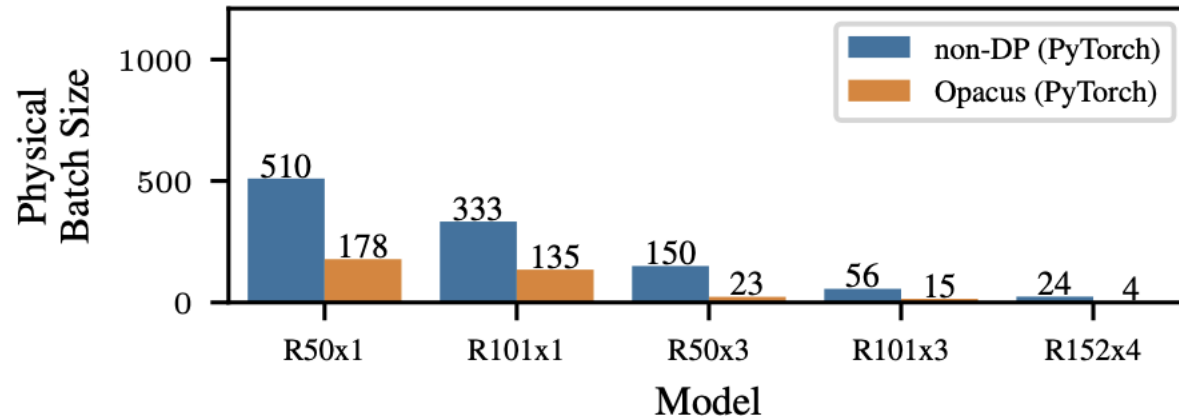
(3) Small noise

As we train for more epochs, we spend more of our privacy budget. For a fixed ϵ , this is reflected here by an increasing δ . Once δ reaches the chosen threshold, around 10^{-5} , we stop training.



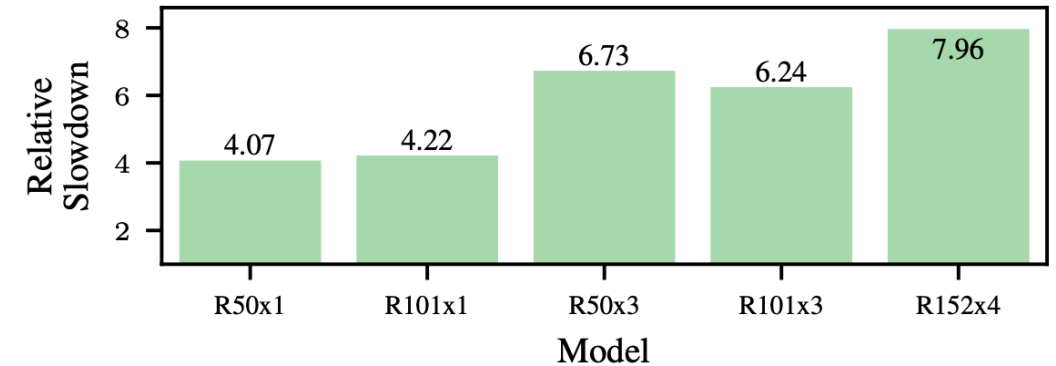
Computational cost of privacy in DP-SGD

- Gaussian Noise addition: Convergence slows down
- Per-sample gradient clipping: Must store per-sample gradient in memory, memory required increases, **throughput** (#training samples processed per sec) reduces



Maximum achievable batch size of ResNet on A100

E.g., **R50x3** is a ResNet-50 architecture whose layers are approximately **3× wider** than the standard ResNet-50



Ratio of mean throughputs between non-private and private baseline

<https://arxiv.org/pdf/2406.17298v3>



DP-SGD for LLMs

- The feasibility of applying DP-SGD to an LLM depends on the training stage
- Pre-training from scratch: very large performance drop, often impractical
- Fine-tuning a pretrained model: small accuracy loss, works well at $\epsilon = 3 - 8$

Table 1: Comparing DP pre-training and DP fine-tuning.

	pre-training	fine-tuning
Dataset size	large	small
Training iterations (amount of compute)	large	small
Trainable model parameters	100%	0.1 ~ 100%
Major cause of performance degradation	DP noising	DP clipping

WHY?

Pretraining: huge numbers of optimization steps -> Privacy loss accumulates-> Gaussian noise can become a major obstacle to learning

Finetuning: Some examples may produce large and informative gradients as the pretrained model needs to adapt to a new task/domain. Aggressive clipping can suppress these useful updates.

https://proceedings.neurips.cc/paper_files/paper/2024/file/ac04e54e0a2d1927d60709019e4e7870-Paper-Conference.pdf



Why pre-training with DP is hard

- DP-SGD changes each parameter update in two ways:
 - Clips each sample's gradient (clipping factor c)
 - Adds Gaussian noise (noise scale σ)

$$\tilde{g} = \frac{1}{B} \left[\sum_i \text{clip}(g_i, C) + \mathcal{N}(0, \sigma^2 C^2 I) \right]$$

- These modifications introduce an additional penalty in the expected improvement of training loss
- This penalty is called the **decelerator** **decelerator:** $\frac{\sigma^2 \text{tr}(\mathbf{H})}{Bc^2}$

<https://arxiv.org/pdf/2402.18752>



Why pre-training with DP is hard?

- Overall goal during training is to minimize expected loss over data

$$\min_{\mathbf{w} \in \mathbb{R}^d} L(\mathbf{w}) = \mathbb{E}_x [L(\mathbf{w}, x)]$$

Per sample gradient $\mathbf{g}_i(\mathbf{w}) := \frac{\partial L(\mathbf{w}, x_i)}{\partial \mathbf{w}} \in \mathbb{R}^d$

Oracle gradient $G(\mathbf{w}) := \frac{\partial \mathbb{E}_x [L(\mathbf{w}, x)]}{\partial \mathbf{w}} \in \mathbb{R}^d$

Oracle hessian $\mathbf{H}(\mathbf{w}) := \frac{\partial^2 \mathbb{E}_x [L(\mathbf{w}, x)]}{\partial \mathbf{w}^2} \in \mathbb{R}^{d \times d}$

Assumption: Per-sample gradients are IID with no restriction on the covariance structure

$$\mathbb{E}[\mathbf{g}_i(\mathbf{w})] = \mathbf{G}(\mathbf{w}), \quad \text{Cov}(\mathbf{g}_i(\mathbf{w})) = \Sigma(\mathbf{w})$$

<https://arxiv.org/pdf/2402.18752>



Why pre-training with DP is hard?

For DP optimizers

$$\mathbf{g} = \frac{\sum_{i \in \mathcal{B}} C_i \mathbf{g}_i + \sigma \mathcal{N}(0, \mathbf{I}_d)}{B} \approx \frac{c \sum_{i \in \mathcal{B}} \mathbf{g}_i + \sigma \mathcal{N}(0, \mathbf{I}_d)}{B} \quad \text{Eq (1)}$$

Update rule

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{g}_t$$

By using second-order Taylor approximation, we get

$$L(\mathbf{w}) - L(\mathbf{w} - \eta \mathbf{g}) \approx \eta \mathbf{G}^\top \mathbf{g} - \frac{\eta^2}{2} \mathbf{g}^\top \mathbf{H} \mathbf{g}.$$

By taking expectation on RHS, expected per iteration loss improvement

$$\Delta L := \eta \mathbf{G}^\top \mathbb{E}[\mathbf{g}] - \frac{\eta^2}{2} (\text{tr}(\mathbf{H} \text{Cov}(\mathbf{g})) + \mathbb{E}[\mathbf{g}]^\top \mathbf{H} \mathbb{E}[\mathbf{g}]) \quad \text{Eq (2)}$$

<https://arxiv.org/pdf/2402.18752>



Why pre-training with DP is hard?

Taking expectation and covariance of Eq 1 with IID assumption

$$\mathbb{E}[\mathbf{g}] = c\mathbf{G} \qquad \text{Cov}(\mathbf{g}) = c^2\mathbf{\Sigma}/B + \sigma^2/B^2$$

Substituting into Eq 2

$$\Delta L_{\text{priv}}(\eta, B) := \eta c \mathbf{G}^\top \mathbf{G} - \frac{\eta^2}{2} \left(c^2 \mathbf{G}^\top \mathbf{H} \mathbf{G} + \frac{c^2 \text{tr}(\mathbf{H} \mathbf{\Sigma})}{B} + \frac{\sigma^2 \text{tr}(\mathbf{H})}{B^2} \right)$$

The final term is called the **decelerator**, which isn't present in standard training ($\sigma = 0, c = 1$)



Why pre-training with DP is hard?

- $\text{tr}(H)$ measures the curvature of the loss surface at the current weights
- Pre-training starts with random-initialization, sharply curved surface, $\text{tr}(H)$ is large, noise compounds the effect
- An already pre-trained model, has a comparatively flat region, so $\text{tr}(H)$ is smaller



VaultGemma: A Differentially Private Gemma Model

VaultGemma Team^{1, 2}

¹Google Research, ²Google DeepMind

We introduce VaultGemma 1B, a 1 billion parameter model within the Gemma family, fully trained with differential privacy. Pretrained on the identical data mixture used for the Gemma 2 series, VaultGemma 1B represents a significant step forward in privacy-preserving large language models. We openly release this model to the community.

<https://arxiv.org/pdf/2510.15001>



VaultGemma: How it was pretrained

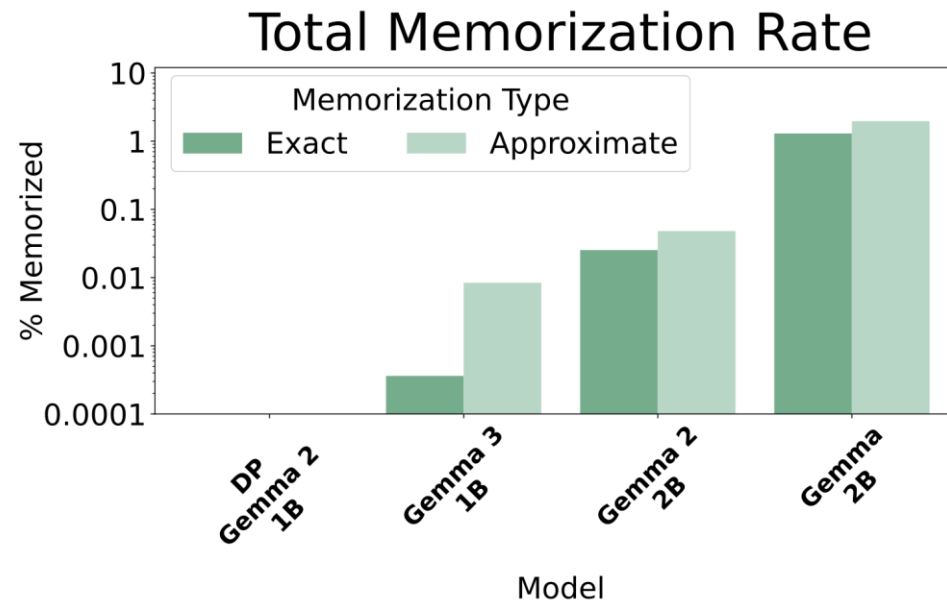
- A very large batch size to reduce decelerator
- Short sequence length (~1024 tokens) to free up compute for large batch size
- Huge compute infrastructure with 2048 TPUv6e chips

Parameters	1B
Iterations	100,000
Expected Batch Size	517,989
Noise Multiplier	0.6143481
ϵ	2.0
δ	$1.1e^{-10}$



VaultGemma: No detectable memorization

- Test: feed the model a 50-token prefix of a training example, check if it reproduces the next 50 tokens.
- Every non-private Gemma model reproduces some training data, VaultGemma produces none



VaultGemma: The utility cost

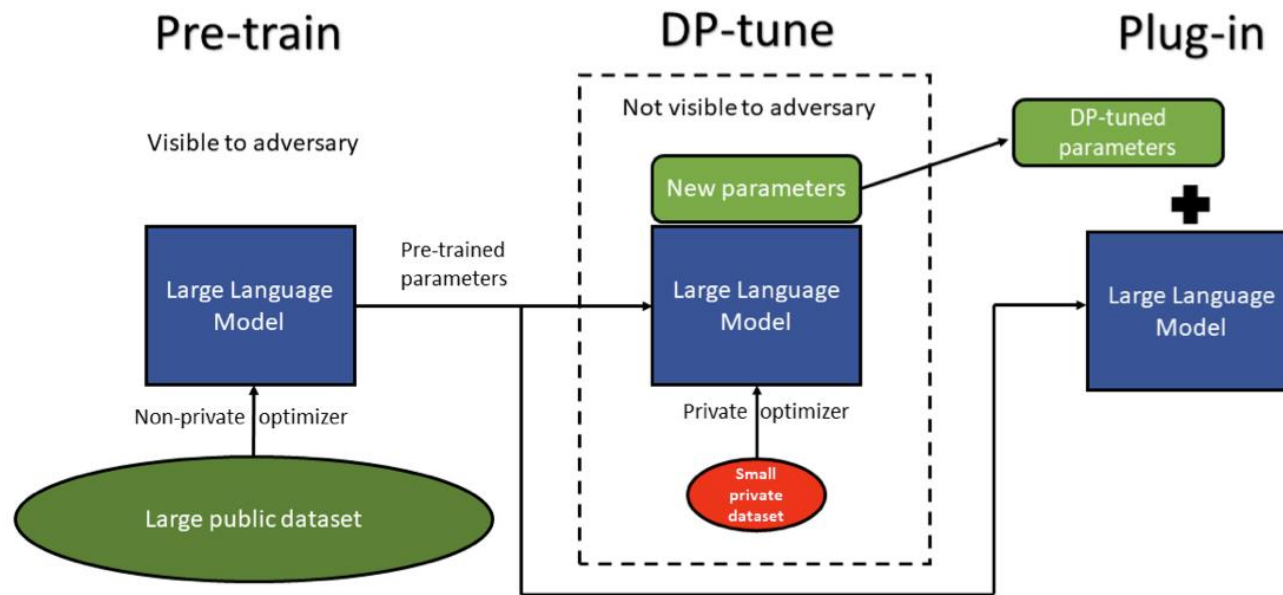
- On standard benchmarks, VaultGemma performs close to GPT-2 1.5B an almost 6-year older model
- A modern non-private Gemma model of same size performs much better.

Model	ARC-C 0 shot	ARC-E 0 shot	HellaSwag 0 shot	PIQA 0 shot	SIQA 0 shot	BoolQ 0 shot	TriviaQA 5 shot
VaultGemma 1B	26.45	51.78	39.09	68.00	46.16	62.04	11.24
Gemma3 1B (PT)	38.31	71.34	61.04	77.37	49.28	68.75	39.75
GPT-2-1.5B	39.78	51.10	47.91	70.51	-	61.80	6.00



Where is DP usually applied?

- Standard practice applies DP optimizers during fine-tuning, on top of a public pretrained model
- Loss landscape is already flat after pre-training, decelerator term is small

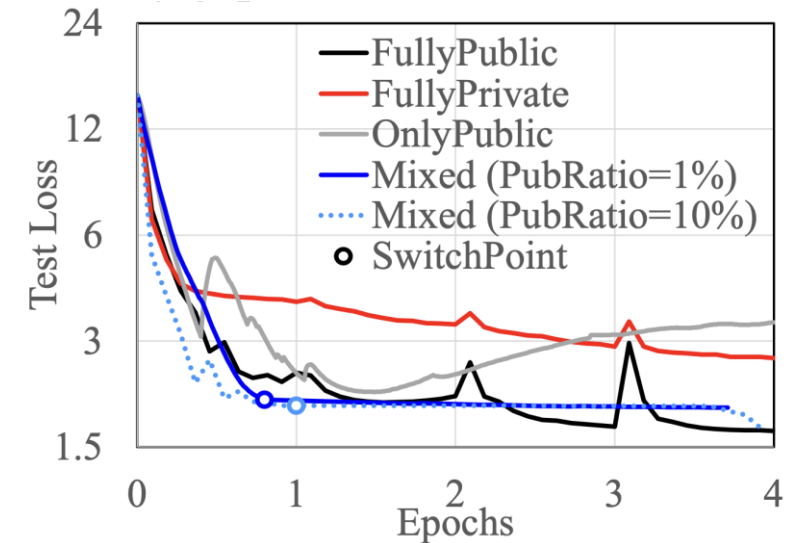


<https://openreview.net/pdf?id=Q42f0dfjECO>



A middle ground: DP continual pre-training

- Bu et al. [NeurIPS'24] suggest a middle ground between entirely pre-training with DP and non-private pre-training.
- Start pre-training on public data with no DP, then switch on DP-SGD to continue pre-training on private data.
- The public phase moves the model into a flat region in the loss landscape, so decelerator is already small when noise is applied.
- With $\sim 10\%$ public data this method reaches near non-private accuracy while still protecting private pre-training data.



GPT2-small pre-training on CodeParrot dataset
($\epsilon = 8$ for mixed and private)

CodeParrot: Github dataset with $\sim 20\text{M}$.py files

<https://arxiv.org/pdf/2402.18752>



DP fine-tuning: Protect only the downstream data

- Most cost-effective strategy: skip private pre-training, apply DP only when fine-tuning
- Pre-training corpus is no longer protected, but the computational overhead is almost eliminated

Metric	DP Guarantee	Gaussian DP + CLT	Compose tradeoff func.	full	LoRA	Method		top2	retrain
BLEU	$\epsilon = 3$	$\epsilon \approx 2.68$	$\epsilon \approx 2.75$	61.519	58.153	47.772	58.482	25.920	15.457
	$\epsilon = 8$	$\epsilon \approx 6.77$	$\epsilon \approx 7.27$	63.189	63.389	49.263	58.455	26.885	24.247
	non-private	-	-	69.463	69.682	68.845	68.328	65.752	65.731
ROUGE-L	$\epsilon = 3$	$\epsilon \approx 2.68$	$\epsilon \approx 2.75$	65.670	65.773	58.964	65.560	44.536	35.240
	$\epsilon = 8$	$\epsilon \approx 6.77$	$\epsilon \approx 7.27$	66.429	67.525	60.730	65.030	46.421	39.951
	non-private	-	-	71.359	71.709	70.805	68.844	68.704	68.751

GPT-2 on the E2E dataset (table-to-text generation)

<https://arxiv.org/pdf/2110.05679>



Private fine-tuning favours larger models

- For prior DP work in areas like image classification with CNNs, the traditional belief was smaller models are better for private learning.
- Accuracy of privately trained models typically degraded with increase in number of parameters due to DP noise being injected across every parameter, causing cumulative noise to drown out actual signal.
- [Yu et al. \[ICLR'22\]](#) found the reverse for fine-tuning, scaling GPT-2 up narrows the gap to non-private.

Model	BLEU (DP)	BLEU (non-private)	Drop due to privacy
GPT-2-Medium	42.0	47.1	5.1
GPT-2-Large	43.1	47.5	4.4
GPT-2-XL	43.8	48.1	4.3

Fine-tuning results on DART dataset (table-to-text generation)

DP params: $\epsilon = 6.8$, $\delta = 1e-5$



Making it practical: Ghost clipping

- Naïve DP-SGD clips each example's weight gradient ($\nabla_W L_i$), requiring one full $|W|$ -sized gradient matrix per example, leading to significant memory overhead.
- Recall, for clipping, we only need the per-example gradient norm, not the gradient itself.

$$\bar{\mathbf{g}}_t(x_i) \leftarrow \mathbf{g}_t(x_i) / \max\left(1, \frac{\|\mathbf{g}_t(x_i)\|_2}{C}\right)$$

- For a linear layer $\nabla_W \mathcal{L}_i = g_i^\top a_i$ where a_i = layer input, g_i = gradient w.r.t layer output

$$\|\nabla_W \mathcal{L}_i\|_F^2 = \text{vec}(a_i a_i^\top)^\top \text{vec}(g_i g_i^\top)$$

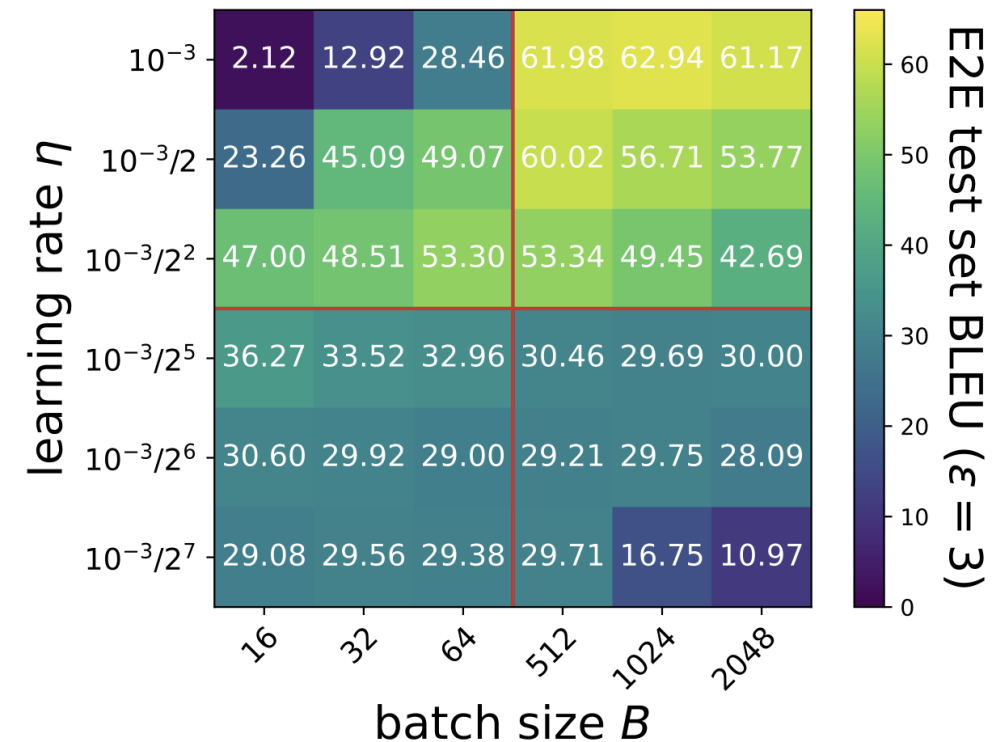
- Then one backward pass on the re-weighted loss $\sum_i c_i L_i$ produces the summed, clipped gradient directly

<https://arxiv.org/pdf/2110.05679>



Hyperparameters for DP fine-tuning

- DP fine-tuning is very sensitive to hyperparameters.
- Usually, a large batch size and large learning rate are preferred, opposite of non-private tuning.
- Aligning the downstream task with pre-training objective (e.g. framing a sentiment classification task as text-infilling) further improves performance.



DP-based Text Sanitization

Text Sanitization with Differential Privacy

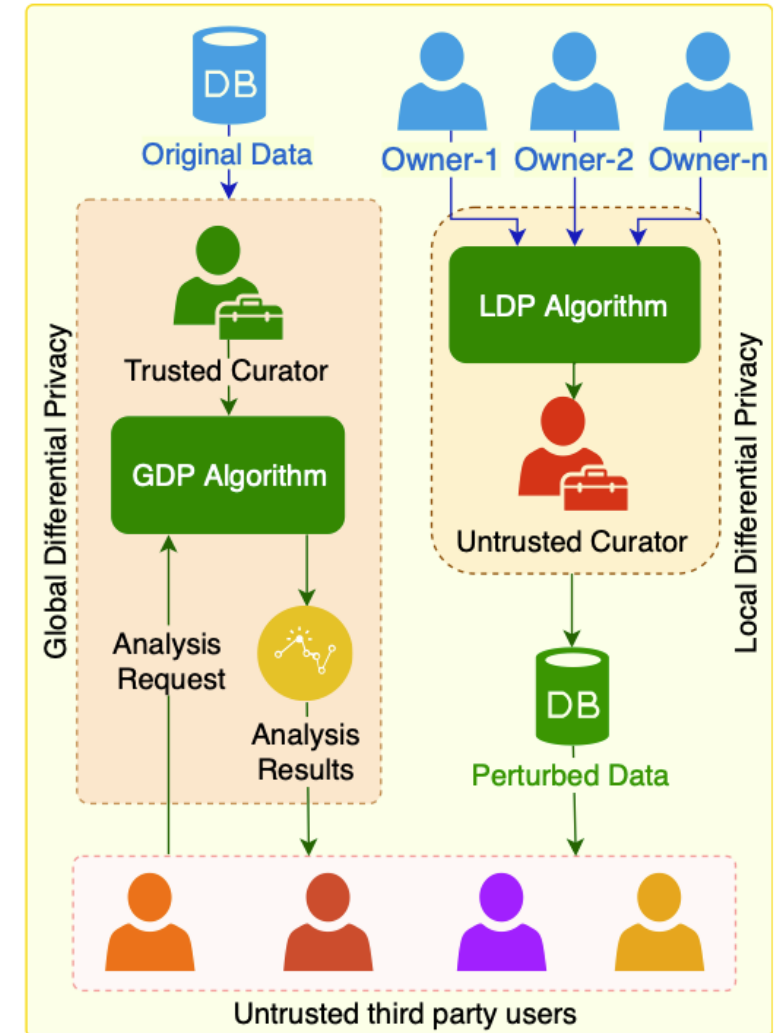
What and why?

What is it?

- A local-DP method that re-writes text by replacing sensitive words with non-sensitive ones.
- Runs on the user side, the model sees only the sanitized text

Motivation

- Training-time DP (DP-SGD) protects the model, not the text the user shares
- We want a format guarantee on the shared text itself



The standard mechanism for token substitution

- Replace each sensitive token by sampling a similar token from a candidate list.
- Sampling uses the exponential mechanism: $\Pr \propto \exp\left(\frac{\epsilon \cdot u}{2\Delta u}\right)$ where u indicates the similarity of the original (x) and replacement token (y) and $\Delta u := \max_{y \in \mathcal{Y}} \max_{x, x' \in \mathcal{X}} |u(x, y) - u(x', y)|$ is the sensitivity of u .
- Two design questions every method must answer:
 - Which token should be labelled sensitive?
 - What candidate list do we sample from?

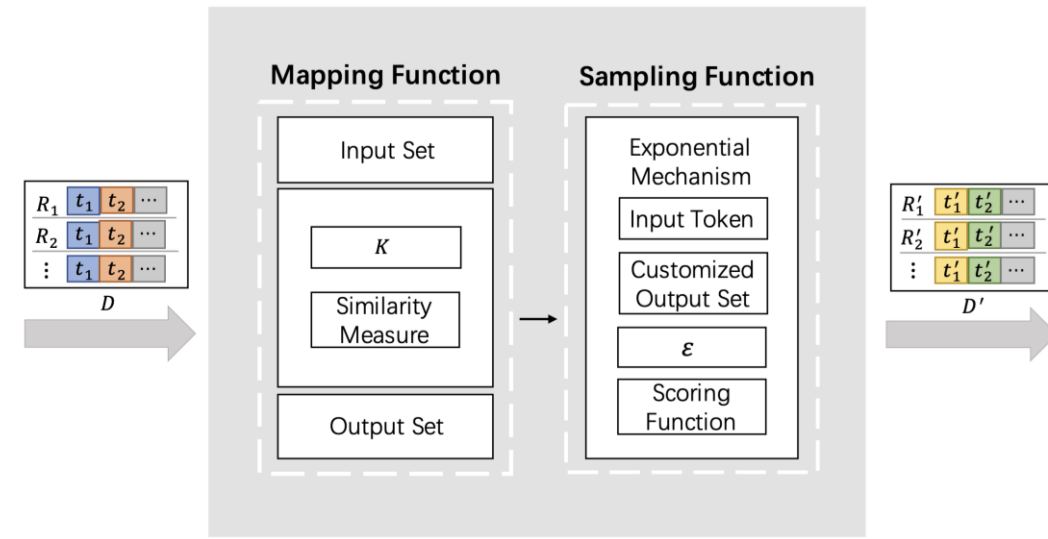


Figure Source: CusText



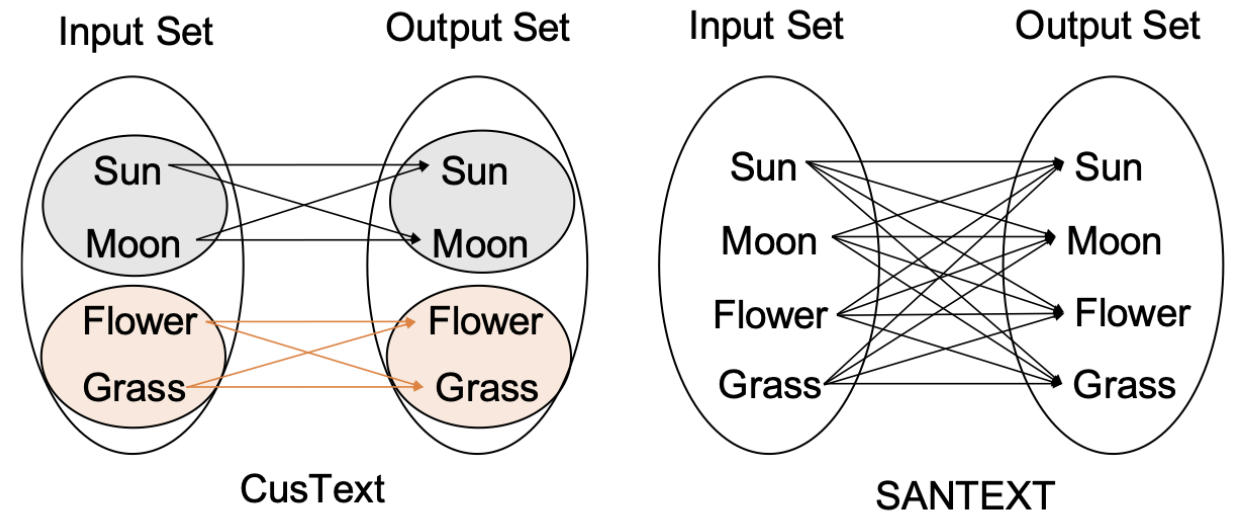
CusText: Candidate list

Idea

- For each sensitive word, fix a list of its K nearest words using semantic distance ($K \approx 20$)
- Sample the replacement only from that list

Why

- Keeps the replacement semantically close, so utility stays high
- Works with any similarity measure, not only metric distances.



CusText+: Which words are sensitive?

- CusText (base method) treats every token as sensitive and perturbs all of them
- CusText+ uses a simple rule: skip a fixed stopwords list (e.g. from NLTK), every other word is considered sensitive.
- **Consequence:** a highly revealing word (e.g. cancer) gets no more protection than an ordinary one.



CusText+: Results

- Privacy is measured using query attack: Repeatedly querying sanitized document until original token is recovered

Token	SANTEXT			CusText (GloVe)				CusText (Counter-Fitting)			
	$\epsilon' = 1$	$\epsilon' = 2$	$\epsilon' = 3$	$\epsilon = 1$	$\epsilon = 2$	$\epsilon = 3$	$\epsilon = 8$	$\epsilon = 1$	$\epsilon = 2$	$\epsilon = 3$	$\epsilon = 8$
she	2350	35	4	1000	200	80	5	5500	1000	320	4
car	1300	14	1	1220	250	90	6	420000	90000	31000	3200
alice	1550	20	3 ✓	1190	240	100	6	1700	360	120	9
happy	3200	55	4	1490	290	110	8	320000	55000	21500	1500
Accuracy	0.4959	0.5799	0.7958	0.6985	0.7172	0.7029	0.8155	0.7117	0.7370	0.7298	0.7957

Privacy: Number of queries needed to recover mentioned token

Utility: Accuracy on SST-2 (sentiment analysis task)



CusText: Drawbacks and towards adaptive protection

Drawbacks

- **Simplistic sensitivity rule:** Treats every non-stopword as sensitive, without actually considering privacy risk
- **Static candidate list:** Uses the same replacement candidates for a sensitive token in every run, making the mechanism more predictable and easier to attack.
- **Uniform candidate list size:** Assigns the same number of replacement candidates to every word, regardless of sensitivity.

Can the candidate list be adapted to each word's actual sensitivity instead of using a fixed, uniform size?



DynText: Intuition

- Decide each word's sensitivity from its **semantic density** in the embedding space.
- Rare words (few close neighbors) are more sensitive
- Frequent words (many neighbors) are less sensitive



DynText: Method

- Estimate each word's density under DP using Gaussian mechanism
- Let $f(t) = |\{\hat{t} \in V_t \mid d(t, \hat{t}) \leq \gamma\}|$ be the true density of token t in embedding space
- $F(t) = f(t) + \mathcal{N}(0, \sigma_d^2)$ noisy density after adding gaussian noise
- $\hat{F}(t) = \frac{F(t) - F_{min}}{F_{max} - F_{min}}$ is the normalized density
- Candidate list size $k_t = \max\left(1, \left\lfloor \left(1 - \hat{F}(t)\right) K \right\rfloor\right)$, where K is the maximum list size
 - Low density -> larger replacement list (more noise, more privacy)
 - High density -> smaller replacement list
- Sample replacement from the list using exponential mechanism (discussed in CusText)



DynText: Results

Privacy

- Input inference attack: the adversary uses a pre-trained BERT model to recover the original private text from the sanitized text by masking and predicting each token
- Input inference attack success rate near 0, stable as ϵ grows

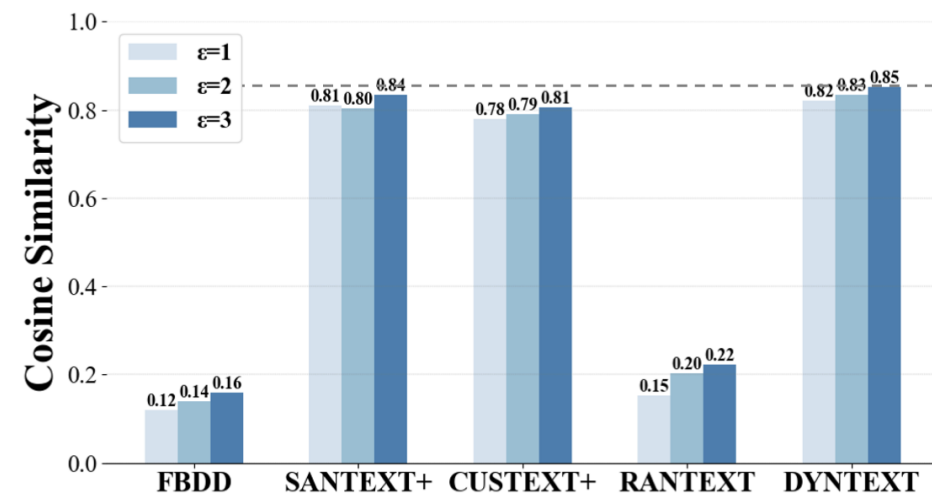
Method	IMDb			20 Newsgroups			PubMedQA		
	$\epsilon = 1$	$\epsilon = 2$	$\epsilon = 3$	$\epsilon = 1$	$\epsilon = 2$	$\epsilon = 3$	$\epsilon = 1$	$\epsilon = 2$	$\epsilon = 3$
SANTEXT+	0.97143	0.97143	0.97143	0.01136	0.04735	0.18939	0.02667	0.11000	0.25556
CUSTEXT+	0.39778	0.38778	0.37333	0.31439	0.32955	0.36237	0.27333	0.26333	0.27111
RANTEXT	0.00243	0.01160	0.02439	0.00000	0.00192	0.00637	0.00000	0.00333	0.00222
DYNTXT	0.00008	0.00008	0.00007	0.00000	0.00010	0.00009	0.00000	0.00001	0.00002

Attack success rate on input inference attack



DynText: Results

- MAUVE: similarity between LM generated text and human authored target continuation
- Coherence: cosine similarity between text and continuation
- Close to non-private GPT-4 upper bound with respect to utility
- Replacement stays closer to original word (less semantic loss)



Method	IMDb						20 Newsgroups						PubMedQA					
	MAUVE			Coherence			MAUVE			Coherence			MAUVE			Coherence		
GPT-4	0.258			0.599			0.228			0.601			0.315			0.737		
Vicuna-7B	0.094			0.023			0.180			0.406			0.230			0.609		
	$\epsilon=1$	$\epsilon=2$	$\epsilon=3$	$\epsilon=1$	$\epsilon=2$	$\epsilon=3$	$\epsilon=1$	$\epsilon=2$	$\epsilon=3$	$\epsilon=1$	$\epsilon=2$	$\epsilon=3$	$\epsilon=1$	$\epsilon=2$	$\epsilon=3$	$\epsilon=1$	$\epsilon=2$	$\epsilon=3$
FBDD	0.049	0.074	0.062	0.169	0.169	0.172	0.056	0.040	0.043	0.159	0.156	0.157	0.092	0.096	0.078	0.352	0.351	0.352
SANTEXT+	0.205	0.228	0.236	0.403	0.463	0.550	0.115	0.102	0.135	0.373	0.418	0.494	0.219	0.230	0.238	0.595	0.676	0.726
CUSTEXT+	0.225	0.252	0.197	0.588	0.580	0.550	0.153	0.171	0.152	0.557	0.562	0.562	0.183	0.224	0.219	0.693	0.698	0.703
RANTEXT	0.038	0.047	0.054	0.113	0.125	0.128	0.030	0.040	0.047	0.095	0.125	0.132	0.010	0.010	0.010	0.127	0.142	0.151
DYNTEXT	0.241	0.254	0.242	0.589	0.590	0.590	0.183	0.180	0.158	0.578	0.579	0.579	0.271	0.289	0.341	0.727	0.728	0.732



Conclusion

